EXECUTIVE SUMMARY

Technical Exercise — Head of Software Engineering & Solution Architecture

Candidate	*Jacques du Plessis*
Product	*POC Product API intended for Whatsapp Content Distribution Service*
Deployment URL	https://jac-profile-api-demo-bufwb9ekhzcfawh0.uksouth-01.azurewebsites.net/
Repository	https://github.com/jacques-dup/alpha-product-api-demo
Commit SHA	6e4632a6f9ef32744125887586a5b92e78192492

1. USER & PROBLEM

The current Whatsapp Content Distribution service uses hard coded json manifests in order to provide the product details for the supported Alpha Courses we support with it. The user problem I'd like to address is to enable better support and Alpha course rollout by creating a Product API and management portal, where the product information can be configured.

2. SOLUTION

I've built a C# API using our standard stack. The API is authenticated by our own Identity Provider (IDP). The data is stored in a Postgres database. I've also built a Admin portal for the API that can be used to perform full CRUD operations on the data. The whole stack is deployed to our Azure subscription.

3. ARCHITECTURE

1. *A C# API, that also serves as the BFF for the Admin Portal. Hosted as a Azure Web App. It uses a design inspired by the Ports and Adapters pattern in order to simplify development and have clear modules that fit within our Domain Driven design paradigm.*
2. *A Javascript Admin Portal deployed to the wwwroot of the Web App. This uses the React Admin package to swiftly be able to provide a full CRUD experience on top of the API. The package has a standard layout we can leverage, is fully customizable and well documented, which meant that AI was very capable of rolling out any changes I wanted to make.*
3. *A Postgres database for the product data*

4. KEY DECISIONS & TRADE-OFFS

The Data Model was the biggest decision that took the most time to design. We have a legacy system built on Wordpress that is very convoluted and complex. It was difficult to decide how to structure the product data and how to design a simple "replacement" that will serve existing as well as future needs – if we wanted to replace the Wordpress system with a full Courses Service based on this POC. Whether to use our existing IDP stack or not, since we don't fully support Role based Access. Also how much scope to include. I had to reduce the scope enough that the POC is not a 1-1 replacement for the Whatsapp Solution, and will need more work to be a drop in replacement.

5. SECURITY

Safety and Security first approach. I know the capabilities of our IDP platform and was able to keep our Auth flow in mind from the very start. The GET requests all have read scope only. A portal user has write scope. The GET API uses a client credential token – as it would be consumed by another client, while the portal has the full authorization flow with a user login and cookie. We keep all secrets in a key-vault and use dotnet user secrets instead of .env files wherever I can. I make sure that all required .env files are always git-ignored. Since we don't have RBAC completed with IDP I also implemented the allow-list as one of the first mechanisms.

6. TESTING

I write tests as I work. AI also aids with this. All tests, specifically are manually checked to make sure that we are actually testing relevant rules. Currently the back-end has 72.5% coverage. The portal had less of a focus since it is mostly framework code, so it has about 27.5% - with the focus being on functionality. I also do manual testing on my local machine before committing and deploying anything to the hosted environment.

7. KNOWN LIMITATIONS

Summarized from the full dossier – we don't have RBAC. The API is not yet on a 1-1 replacement for the Whatsapp project. We will need additional work specifically in seeding the Database. And the Portal needs a better design – it currently just uses the default React Admin component design. Deployment is also still manual so we'll need to build a pipeline.

8. NEXT STEPS

Evaluation and further research. If we go forward, the data seed, IDP work and Automated deployments will need to be done. Then to have the Whatsapp project adopt it we'll need to build a BFF for it so that it can authenticate with the API.